Machine Learning

Lecture 5: Deep learning for sequences

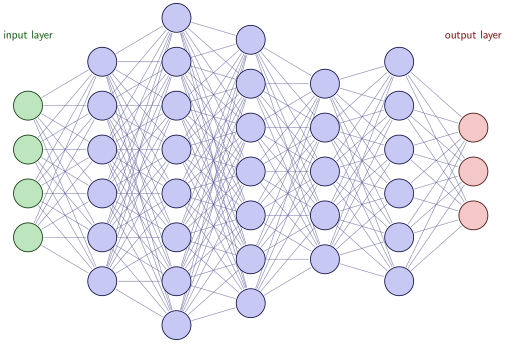
Simon BERNARD
simon.bernard@univ-rouen.fr



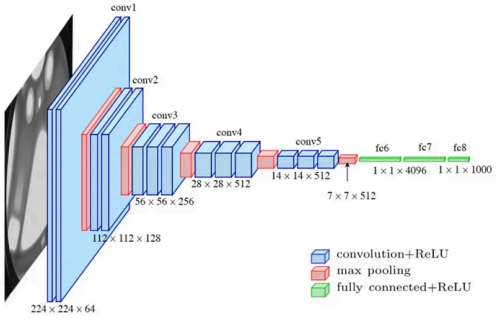
Learning from sequences

Machine learning with sequences

- Fully connected networks: inputs are fixed-size vectors

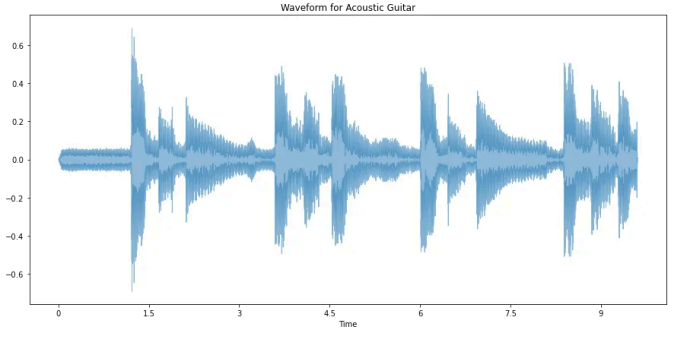


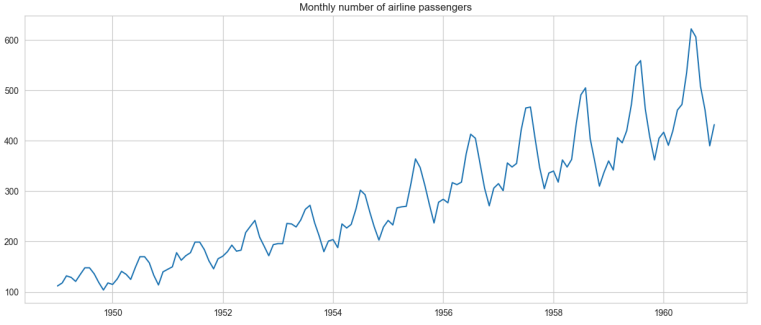
- Convolutional networks: inputs are fixed-size 2D/3D tensors

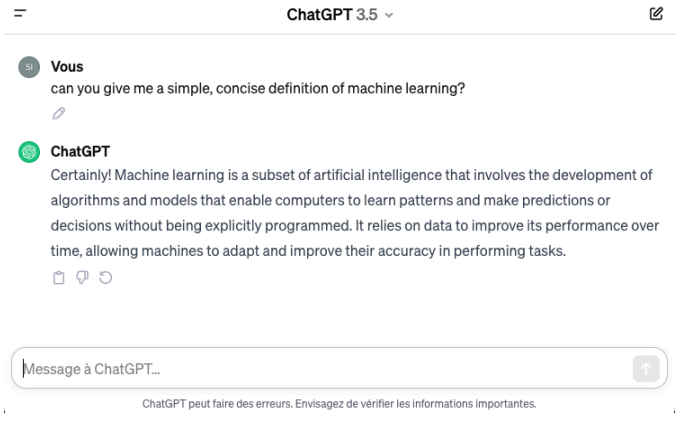


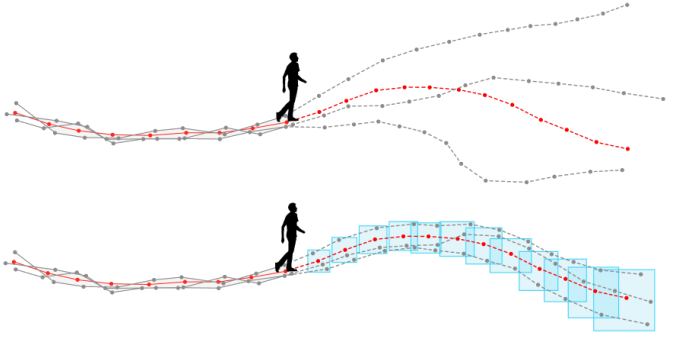
Machine learning with sequences

- Recurrent networks: inputs are variable-length sequences (e.g. text, audio, signals)









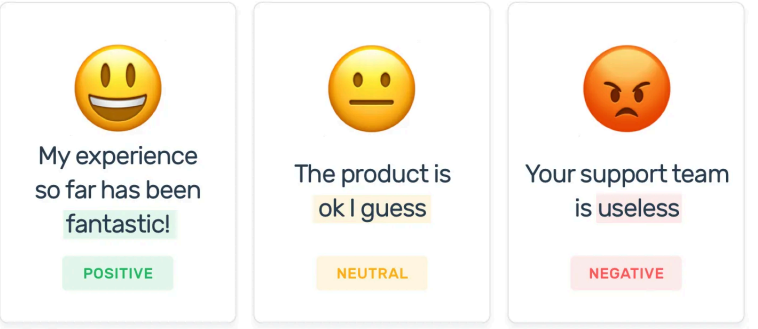
Machine learning with sequences

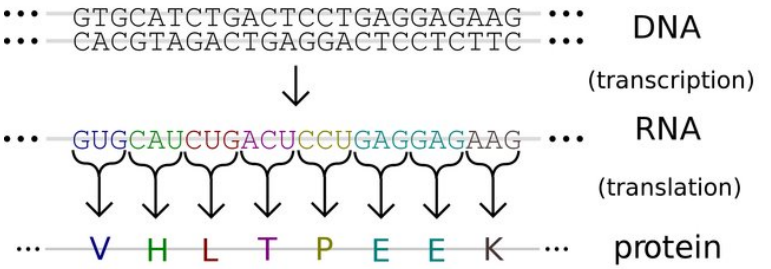
Sequence classification

- Assign a class to a given sequence

$$\text{Sequence} \to \mathbb{N}$$

- Example: sentiment analysis, DNA sequence classification





Machine learning with sequences

Sequence labeling

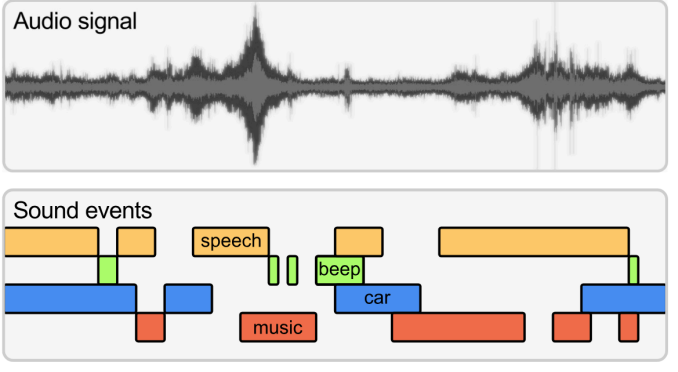
- Assign a class to every item of a given sequence

$$\text{Sequence} \rightarrow \mathbb{N}^T$$

  T being the length of the sequence
- Example: Named-entity recognition, Part-of-speech tagging, audio annotation

When Sebastian Thrun PERSON started at Google ORG in 2007 DATE, few people outside of the company took him seriously. “I can tell you very senior CEOs of major American NORP car companies would shake my hand and turn away because I wasn’t worth talking to,” said Thrun PERSON, now the co-founder and CEO of online higher education startup Udacity, in an interview with Recode ORG earlier this week DATE.

A little less than a decade later DATE, dozens of self-driving startups have cropped up while automakers around the world clamor, wallet in hand, to secure their place in the fast-moving world of fully automated transportation.



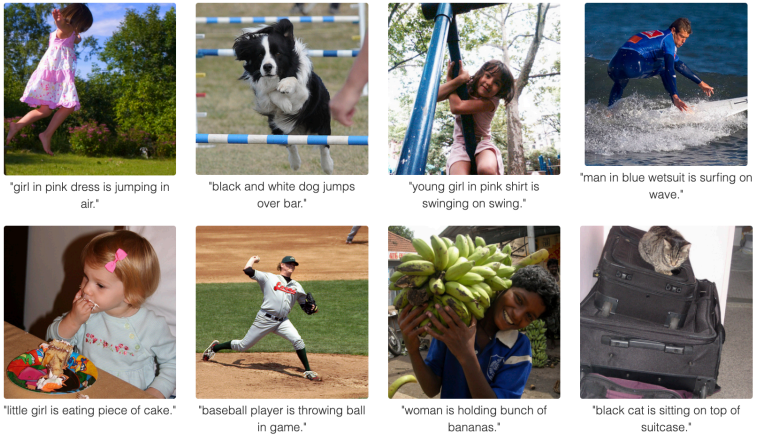
Machine learning with sequences

Sequence synthesis

- Generate a sequence from a given input

$$\mathbb{R}^d \rightarrow \text{Sequence}$$

- Example: Image captioning, Question answering, Music generation
- Note: inputs are considered to be vectors such as image or text embeddings



The first recorded travels by Europeans to China and back date from this time. The most famous traveler of the period was the Venetian Marco Polo, whose account of his trip to "Cambaluc," the capital of the Great Khan, and of life there astounded the people of Europe. The account of his travels, Il milione (or, The Million, known in English as the Travels of Marco Polo), appeared about the year 1299. Some argue over the accuracy of Marco Polo's accounts due to the lack of mentioning the Great Wall of China, tea houses, which would have been a prominent sight since Europeans had yet to adopt a tea culture, as well the practice of foot binding by the women in capital of the Great Khan. Some suggest that Marco Polo acquired much of his knowledge through contact with Persian traders since many of the places he named were in Persian.

How did some suspect that Polo learned about China instead of by actually visiting it?

Answer: through contact with Persian traders

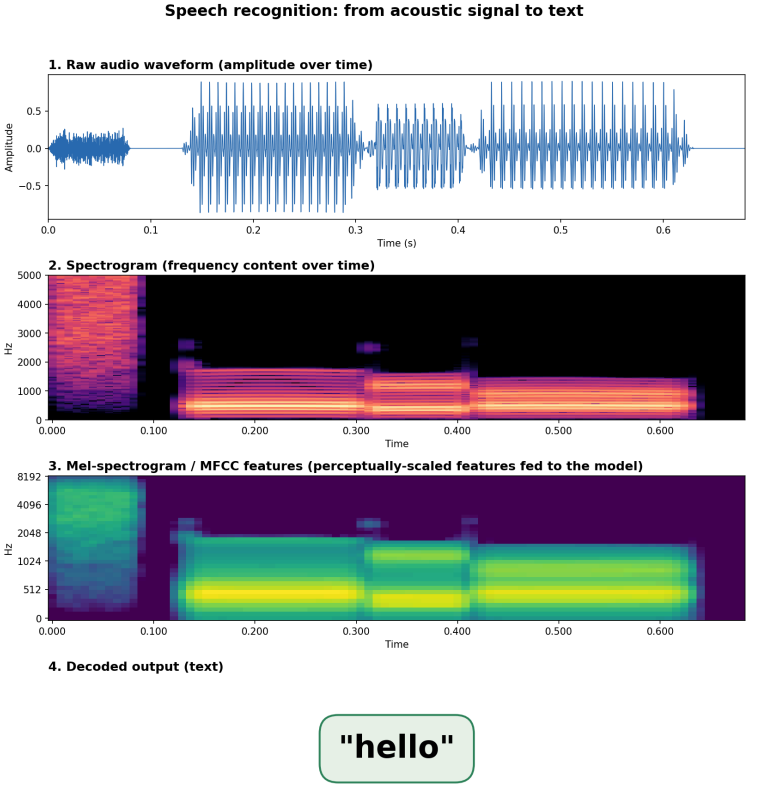
Machine learning with sequences

Sequence to sequence translation

- Generate an output sequence from a given input sequence

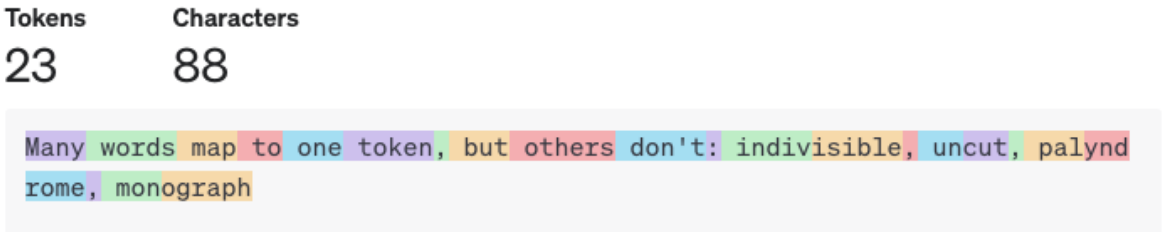
$$\text{Sequence} \rightarrow \text{Sequence}$$

- Example: Speech recognition, Machine translation, Text summarization



What is a sequence?

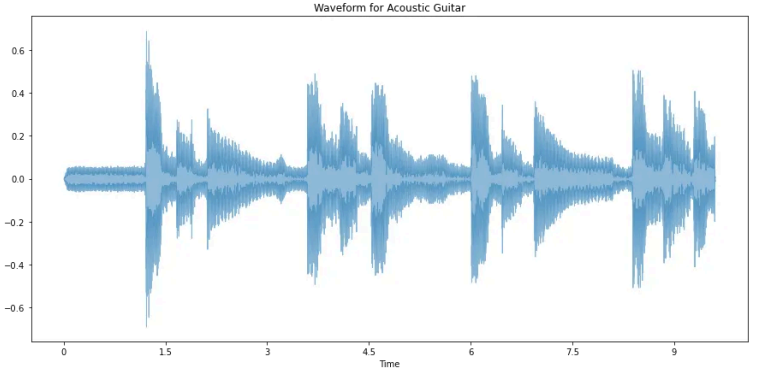
- A T-length sequence is a set of elements $\mathbf{X} = \mathbf{x}_{(1)}, \mathbf{x}_{(2)}, \ldots, \mathbf{x}_{(t)}, \ldots, \mathbf{x}_{(T)}$
- Discrete sequences: $\mathbf{x}_{(t)} \in \mathcal{A}$ where $\mathcal{A}$ is a finite set of symbols (alphabet) and $|\mathcal{A}| = N$
  - Text as sequence of characters: $N \approx 100$ symbols
  - Text as sequence of words: $N \approx 200000$ words or more
  - Text as sequence of tokens (subwords): $N \approx 50000$ tokens

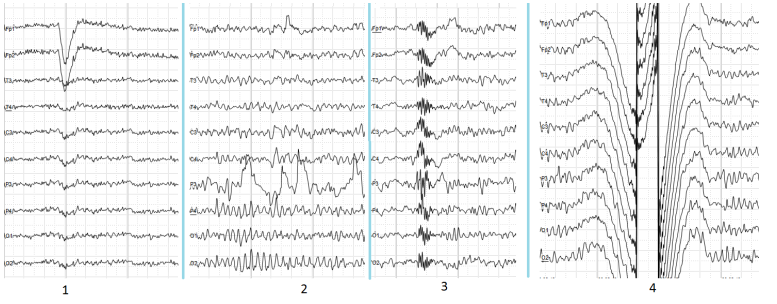


source: OpenAI tokenizer

What is a sequence?

- A T-length sequence is a set of elements $\mathbf{X} = \mathbf{x}_{(1)}, \mathbf{x}_{(2)}, \ldots, \mathbf{x}_{(t)}, \ldots, \mathbf{x}_{(T)}$
- Continuous sequences: $\mathbf{x}_{(t)} \in \mathbb{R}^d$
  - Audio signals: $d = 1$
  - EEG signals: $d = 32, \ldots, 64$





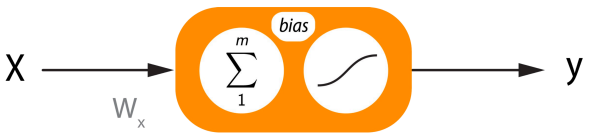
Learning from sequences

- As with CNN, the challenge is to learn a relevant representation of the sequence:
  - Learn to extract local features from the sequence
  - Each layer produce new features with higher-level abstraction
  - Weights are shared across parts of the sequence
- But, we need to model the **temporal dependencies** instead of spatial dependencies
- Implies to model long-range dependencies instead of short-range (local) dependencies

Recurrent neural networks

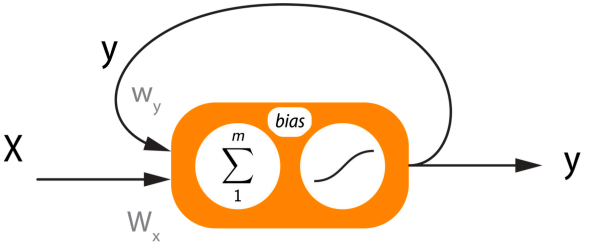
Recurrent neuron

Classical neuron



$$y = \sigma(W_x^T \cdot X + b)$$

Recurrent neuron

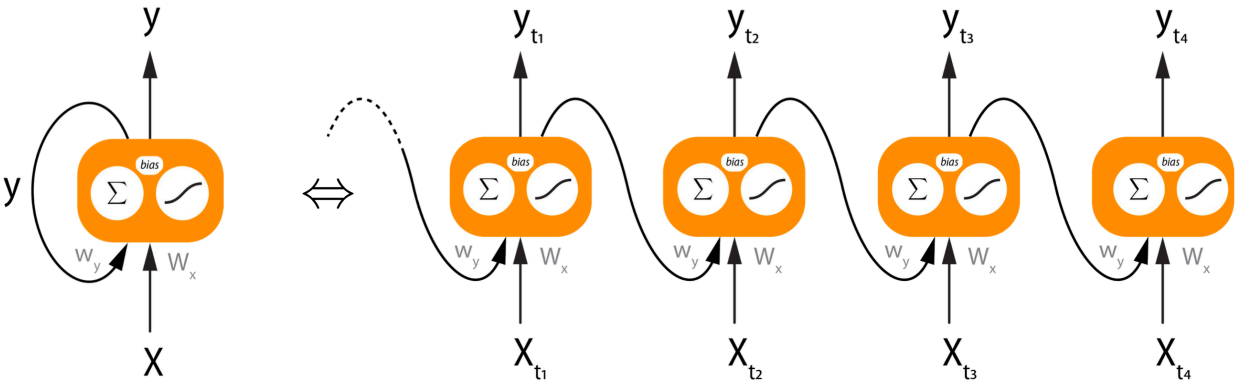


$$y_{(t)} = \sigma(W_x^T \cdot X_{(t)} + w_y \cdot y_{(t-1)} + b)$$

- $\mathbf{x}_{(t)}$ given to the neuron one after the other, following the order in the sequence ($\mathbf{x}_{(1)}$, then $\mathbf{x}_{(2)}$, and so on up to $\mathbf{x}_{(T)}$)
- $\mathbf{y}_{(t)}$, output at time t depends on $\mathbf{x}_{(t)}$ and on $\mathbf{y}_{(t-1)}$

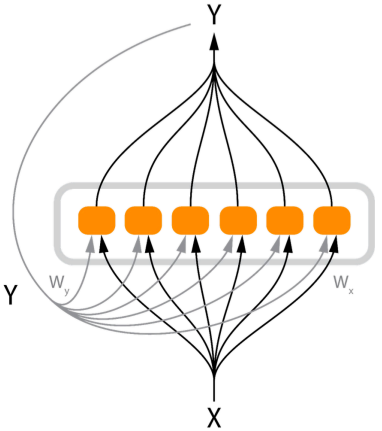
Recurrent neuron

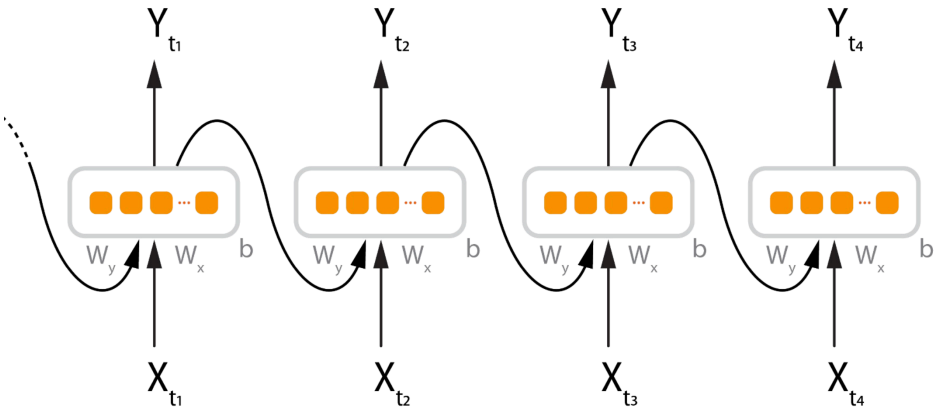
- Same neuron unfolded $\approx T$ classical neurons
- $\mathbf{y}_{(t)}$, reinjected into the neuron at each step, is the **hidden state** of the neuron
- Form of **memory** that allows the neuron to "remember" the previous inputs



Recurrent layer (or cell)

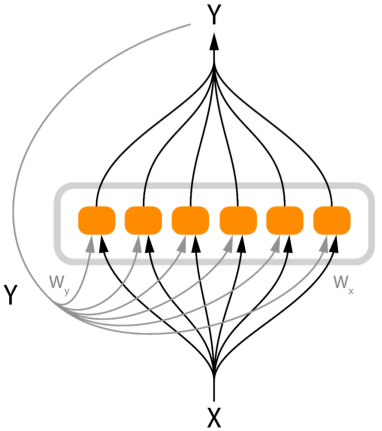
- Same principle transposed to the combination of N neurons
- In that case, neurons are called **units** and their combination is a **cell**
- Output is a vector instead of a scalar (as with a single neuron)

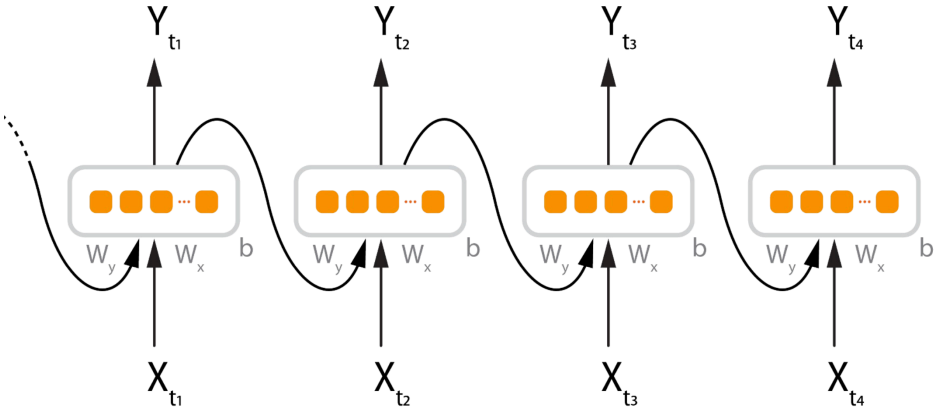




Recurrent layer (or cell)

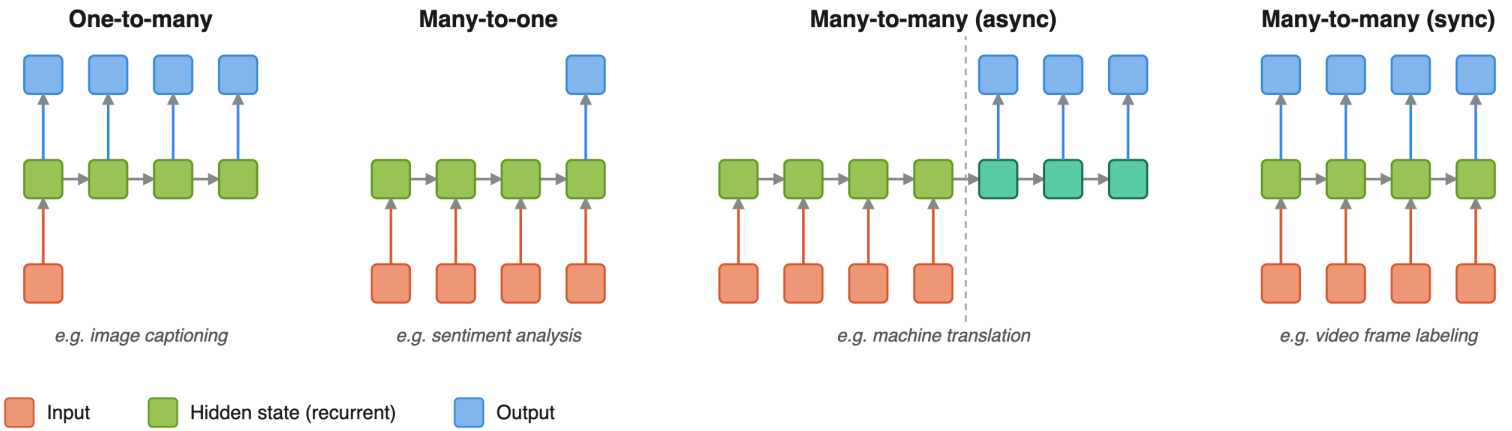
- Output at time t is $\mathbf{y}_{(t)} = \sigma\left(\mathbf{W}_y \mathbf{y}_{(t-1)} + \mathbf{W}_x \mathbf{x}_{(t)} + b\right)$
- $\mathbf{x}_{(t)}$ is a d-sized vector, $\mathbf{y}_{(t-1)}$ is a N-sized vector
- $\mathbf{W}_y$ is a $N \times N$ matrix of weights, $\mathbf{W}_x$ is a $N \times d$ matrix of weights





Recurrent neural networks

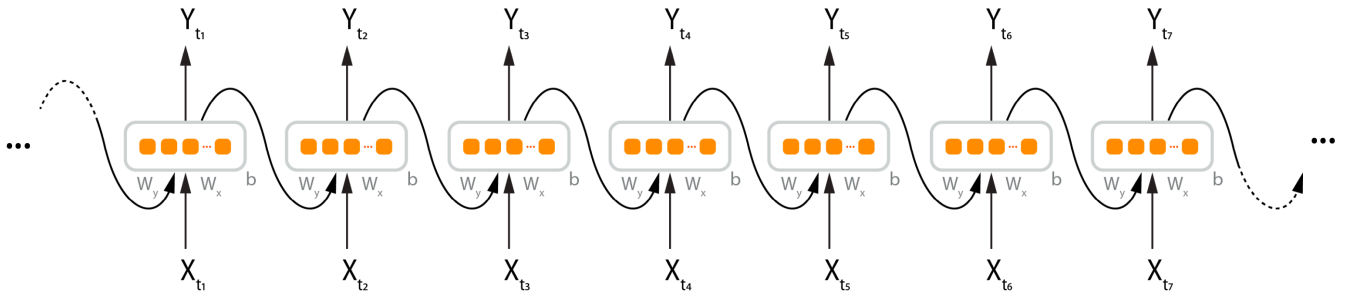
- RNN are made up of **one or several cells of recurrent neurons**
- The architecture depends on the task and the nature of input/output



Recurrent neural networks

Several problems in practice during training:

- Backpropagation through time (BPTT): gradients propagated through the unfolded network

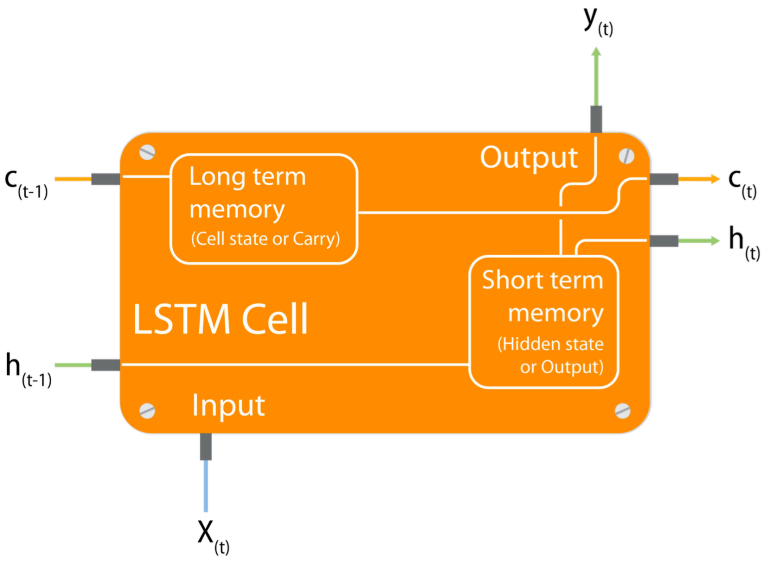


- Can cause **vanishing or exploding gradient problems**
- **Influence of the first inputs can vanish rapidly**: hard to learn long-term dependencies
- Convergence of **training can be very slow**, especially for long sequences

Long story short: **it doesn't work well in practice**

Long Short-Term Memory

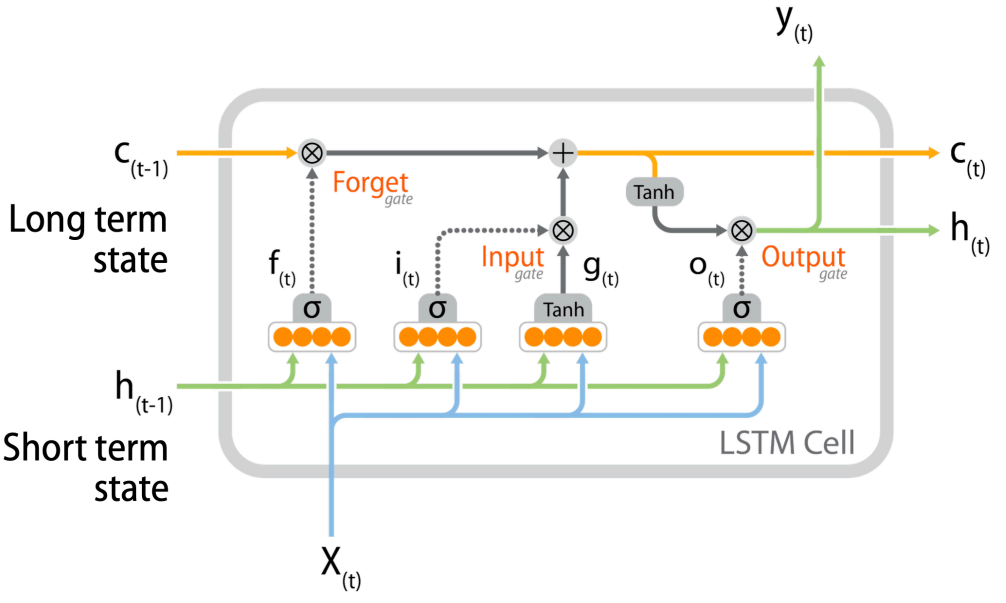
- Solution: use a separate memory cell that can store information for a long time
- The cell is controlled by **gates** that regulate the flow of information in and out of the cell
- This is the principle of **Long Short-Term Memory (LSTM) cells** [1]



- $\mathbf{h}_{(t)}$: hidden state vector (output of the cell)
- $\mathbf{c}_{(t)}$: cell state vector (memory of the cell)
- $\mathbf{h}_{(t-1)}$: previous hidden state vector
- $\mathbf{c}_{(t-1)}$: previous cell state vector

(1) or a more recent and simplified variant, the Gated Recurrent Unit (GRU) cell

Long Short-Term Memory



$$\mathbf{f}_{(t)} = \sigma(\mathbf{W}_{hf}\mathbf{y}_{(t-1)} + \mathbf{W}_{xf}\mathbf{x}_{(t)} + b_f)$$
$$\mathbf{i}_{(t)} = \sigma(\mathbf{W}_{hi}\mathbf{y}_{(t-1)} + \mathbf{W}_{xi}\mathbf{x}_{(t)} + b_i)$$
$$\mathbf{g}_{(t)} = \tanh(\mathbf{W}_{hg}\mathbf{y}_{(t-1)} + \mathbf{W}_{xg}\mathbf{x}_{(t)} + b_g)$$
$$\mathbf{o}_{(t)} = \sigma(\mathbf{W}_{ho}\mathbf{y}_{(t-1)} + \mathbf{W}_{xo}\mathbf{x}_{(t)} + b_o)$$
$$\mathbf{c}_{(t)} = \mathbf{f}_{(t)} \otimes \mathbf{c}_{(t-1)} + \mathbf{i}_{(t)} \otimes \mathbf{g}_{(t)}$$
$$\mathbf{y}_{(t)} = \mathbf{o}_{(t)} \otimes \tanh(\mathbf{c}_{(t)})$$

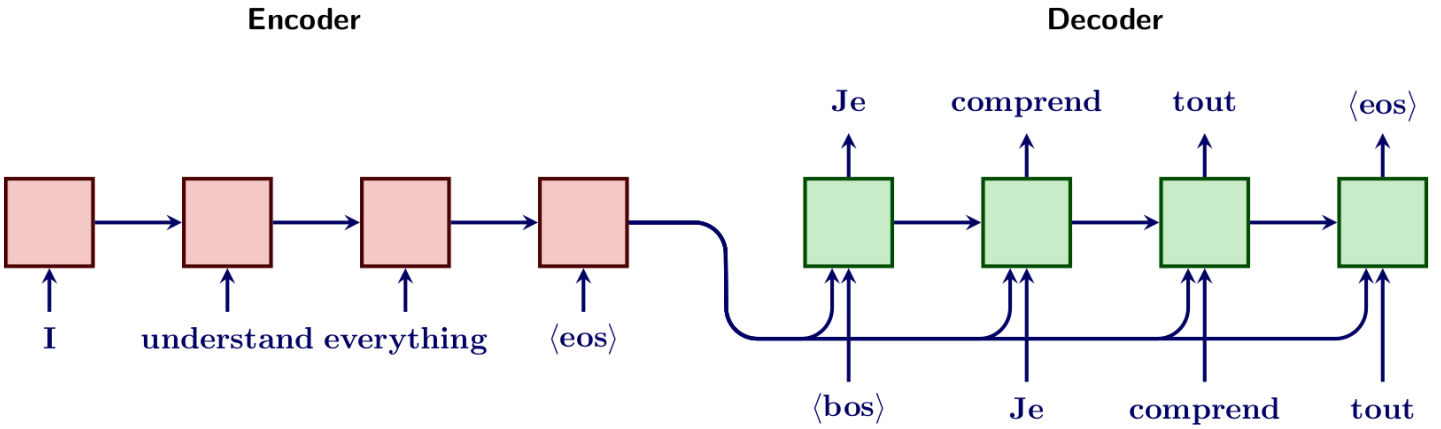
with

- $\mathbf{x}_{(t)} \in \mathbb{R}^d$: input vector
- $\mathbf{f}_{(t)} \in \mathbb{R}^h$: forget gate's activation vector
- $\mathbf{i}_{(t)} \in \mathbb{R}^h$: input gate's activation vector
- $\mathbf{o}_{(t)} \in \mathbb{R}^h$: output gate's activation vector
- $\mathbf{g}_{(t)} \in \mathbb{R}^h$: current entry vector
- $\mathbf{c}_{(t)} \in \mathbb{R}^h$: cell state vector
- $\mathbf{y}_{(t)} \in \mathbb{R}^h$: hidden state / output vector

Attention is all you need

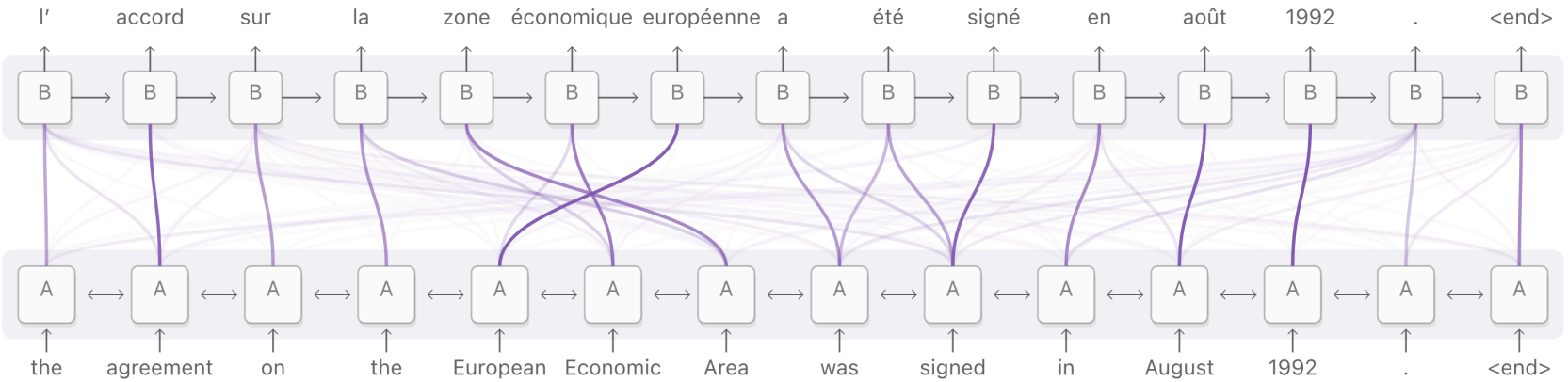
Seq2seq modeling

- Sequence-to-sequence (seq2seq) modeling: generate a sequence from a given input sequence
- Many seq2seq methods are based on **encoder-decoder architectures**
- Encoder: compress the input sequence into a hidden vector representation
- Decoder: generate the output sequence with autoregressive model



Seq2seq modeling

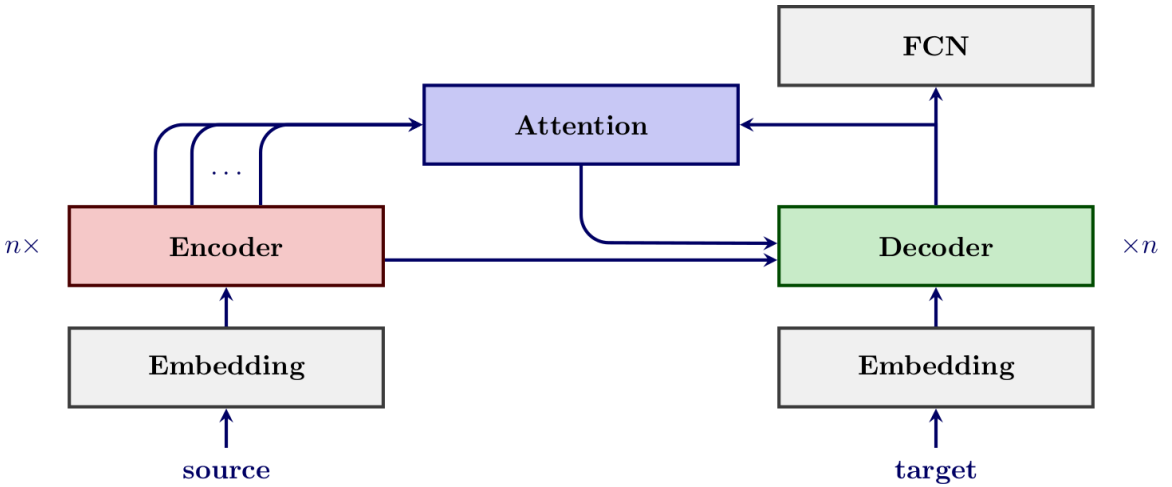
- Problem: the whole input sequence is encoded into a single vector (the last hidden state)
- This **hidden state can not capture all the information of the input sequence**
- Elements of the output sequence do not depend in the same way on all the input elements



source: https://distill.pub/2016/augmented-rnns/

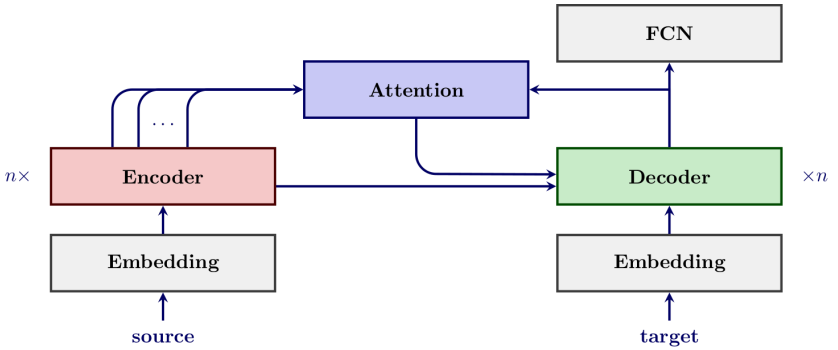
Bahdanau's attention

- Solution: attention mechanism
- Transport information from part of the input sequence to part of the output sequence
- The attention block is designed such that **the focus is dynamic and content-based**



RNN encoder–decoder architecture with the Bahdanau attention mechanism

Bahdanau's attention



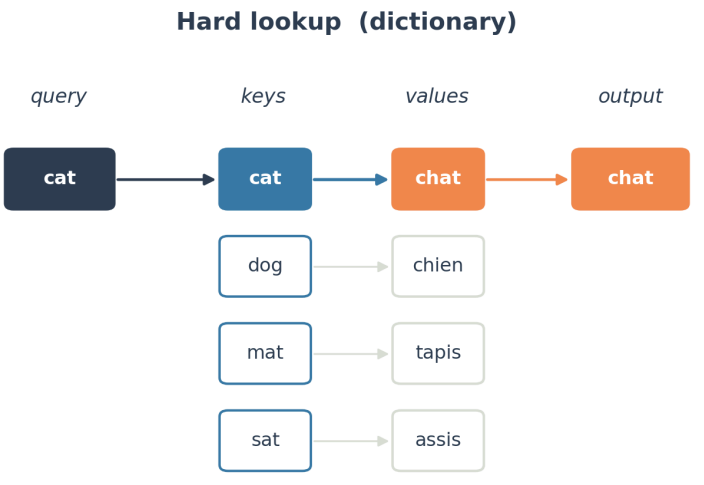
- let $\mathbf{h}_{(t)}$ and $\mathbf{s}_{(t)}$ be the encoder and decoder hidden states at time t
- For generating $\mathbf{s}_{(t')}$, a **context vector** $\mathbf{c}_{(t')}$ is computed as :

$$\mathbf{c}_{(t')} = \sum_{t=1}^{T} \alpha(\mathbf{s}_{(t'-1)}, \mathbf{h}_{(t)})\mathbf{h}_{(t)}$$

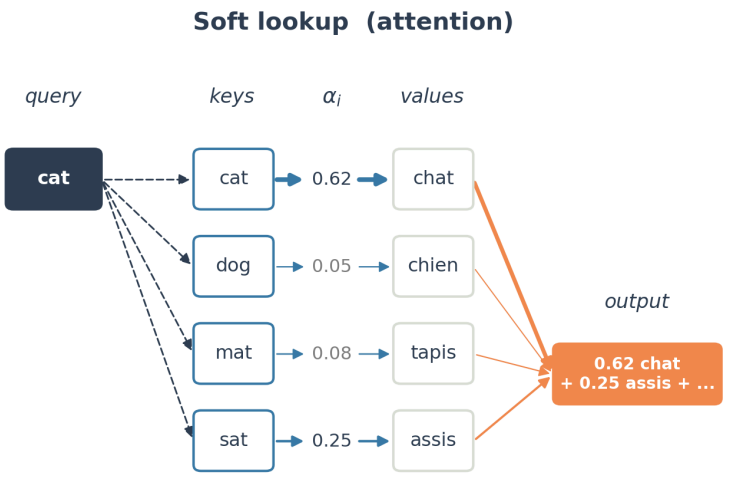
where $\alpha(\mathbf{s}_{(t'-1)}, \mathbf{h}_{(t)})$ measures **the importance of $\mathbf{h}_{(t)}$ for generating $\mathbf{s}_{(t')}$**

Attention as a soft lookup

- Attention is **inspired by information retrieval**
- Dictionary lookup: find the **key** that matches a **query**, and return the corresponding **value**
- Attention: the query is compared to every key, the output is a weighted average of all the values s.t. weights measure how well the query matches each key



exact match → one single value returned



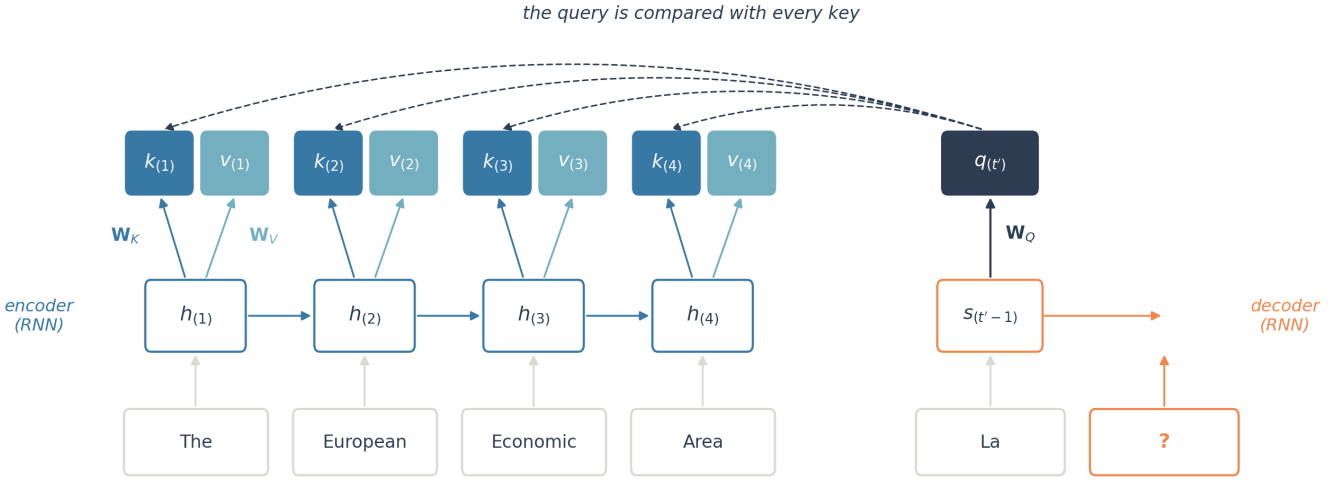
graded similarity → weighted average of all values

Queries, keys and values

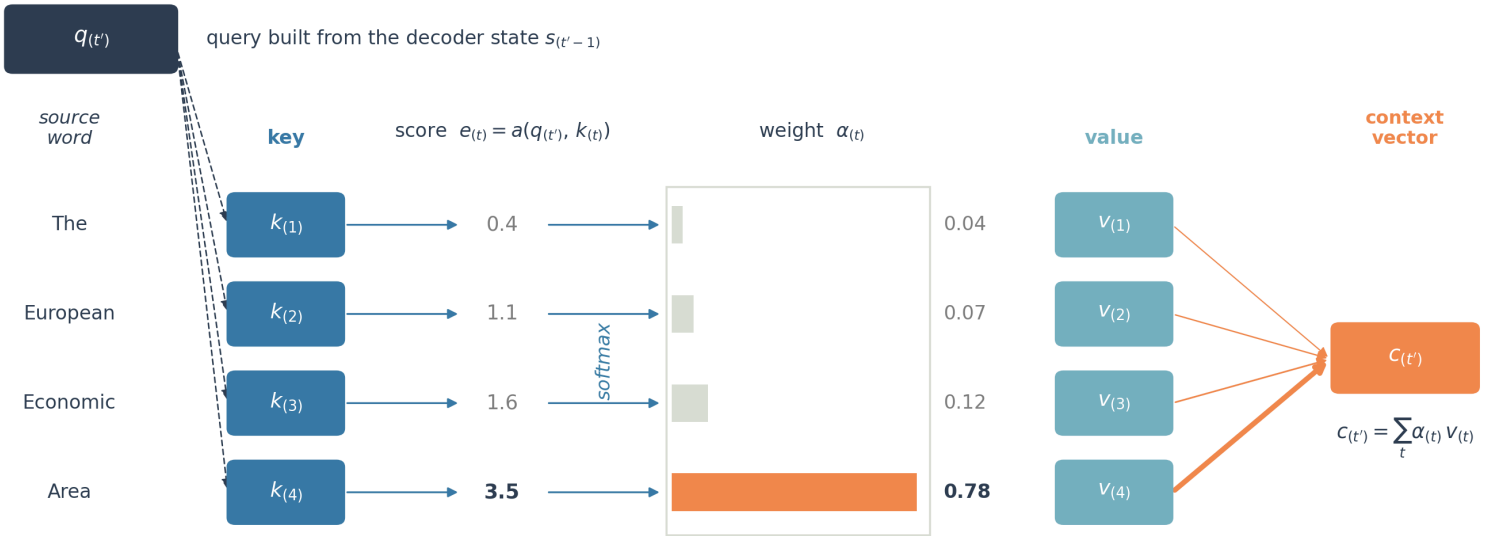
Translating *"The European Economic Area was created in 1992"*:

- The encoder produces one hidden state $\mathbf{h}_{(t)}$ per source word
- Three **learned projections** turn hidden states into the three roles of the lookup

$$\mathbf{q}_{(t')} = \mathbf{W}_Q \, \mathbf{s}_{(t'-1)} \qquad \mathbf{k}_{(t)} = \mathbf{W}_K \, \mathbf{h}_{(t)} \qquad \mathbf{v}_{(t)} = \mathbf{W}_V \, \mathbf{h}_{(t)}$$



Computing the context vector

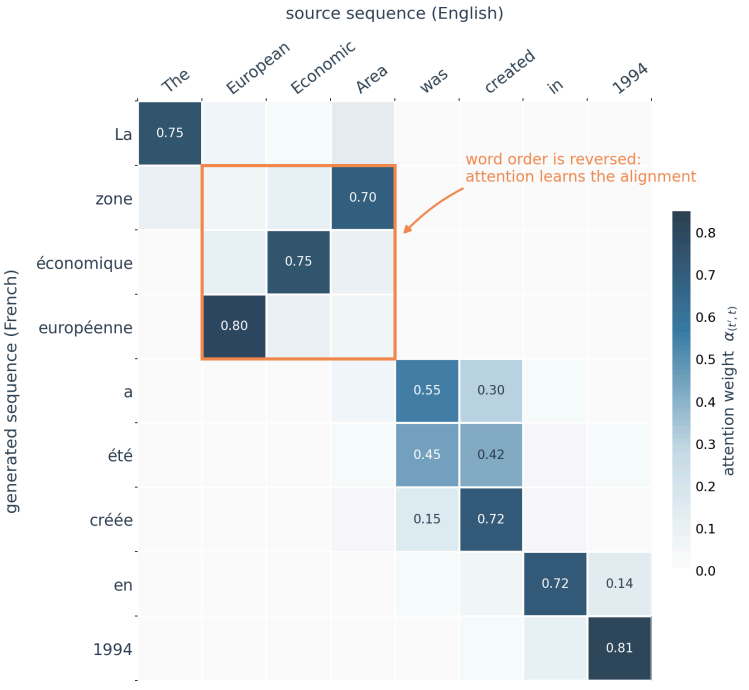


- a is typically the scaled dot product between the query and the key [1]
- α is a softmax over the a values, so that they sum to 1
- Bahdanau's attention is the special case where the three roles are not projected:

$$\mathbf{q}_{(t')} = \mathbf{s}_{(t'-1)} \qquad \mathbf{k}_{(t)} = \mathbf{v}_{(t)} = \mathbf{h}_{(t)}$$

(1) $a(\mathbf{q}_{(t')}, \mathbf{h}_{(t)}) = \frac{\mathbf{q}_{(t')} \cdot \mathbf{k}_{(t)}}{\sqrt{d_k}}$ where d_k is the dimension of the key vectors

Attention learns the alignment



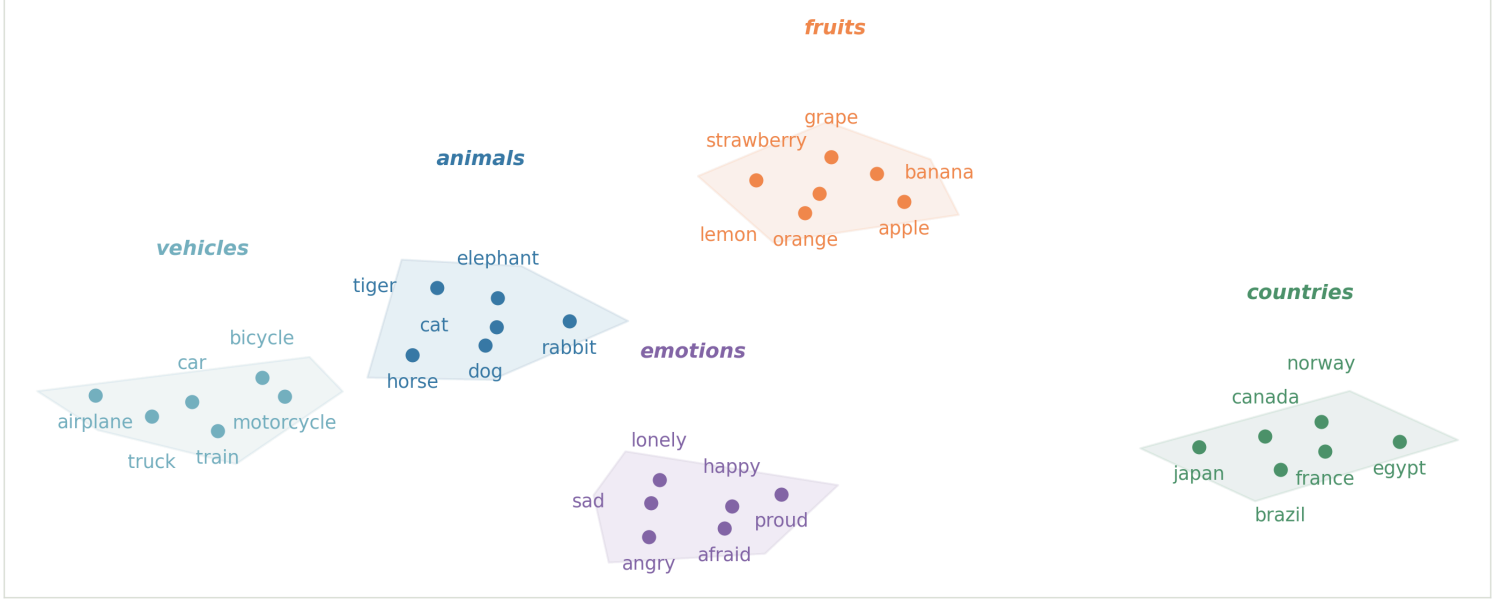
Alignment matrix with the attention weights (illustrative weights, not the output of an actual trained model)

- Nothing tells the model *which* source word to look at: the alignment **emerges from training**
- Handles reordering (*European Economic Area* $\longrightarrow$ *zone économique européenne*) and one-to-many mappings (*created* $\longrightarrow$ *a été créée*)

Transformers

Transformers

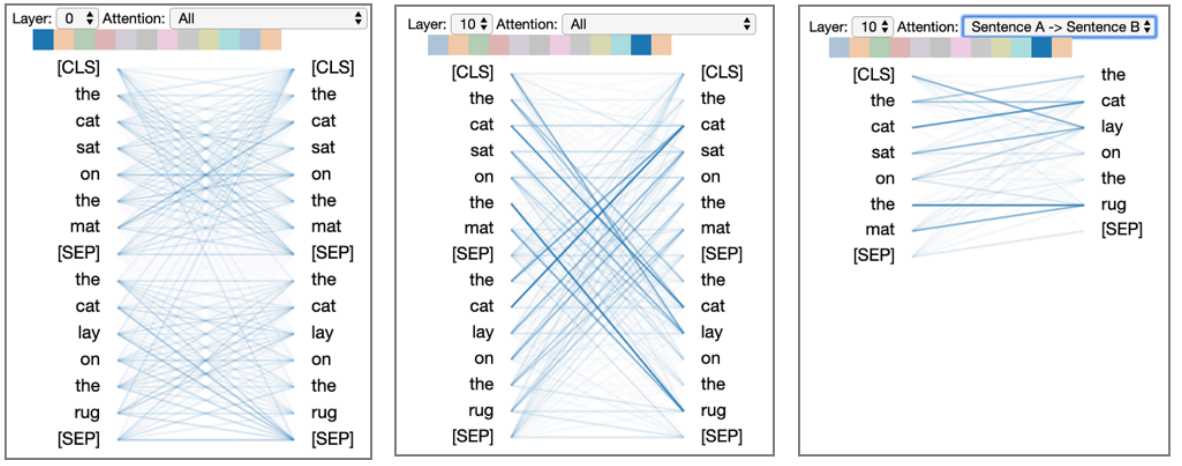
- Encoder-decoder architecture based on attention mechanism
- **Do not use recurrent cells, only attention blocks**
- Exploit attention to build high-level representations of sequences
- E.g. for text, a representation that captures the meaning of the words



2D t-SNE projection of GloVe 100d representation vectors

Transformers

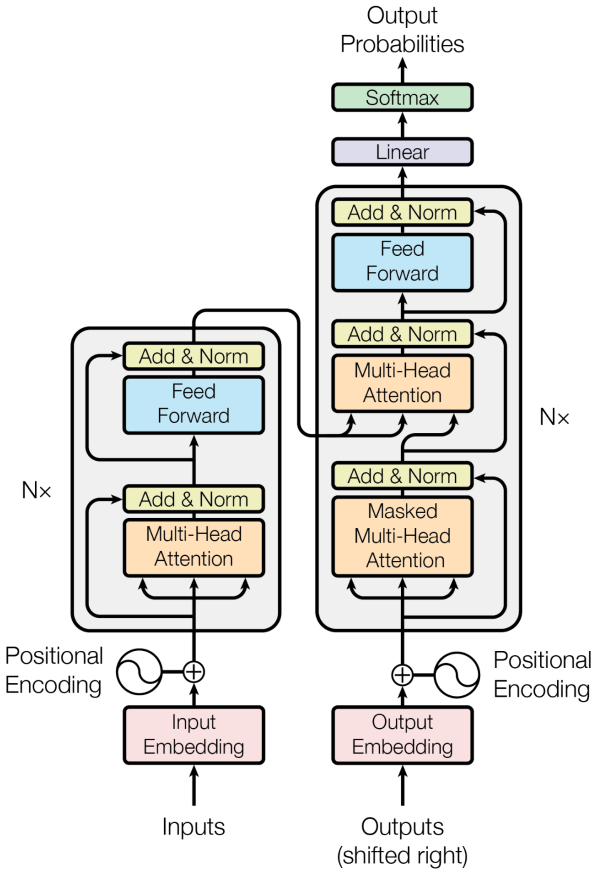
- Key idea is to compute attention between all pairs of elements of a sequence
- Called **self-attention**: queries/keys/values all derived from the same sequence
- This computation [1] is done in several attention "heads" and several layers



source: https://github.com/jessevig/bertviz

(1) Not detailed in this course, but based on multiple parallel operations similar to the previous formulation of attention

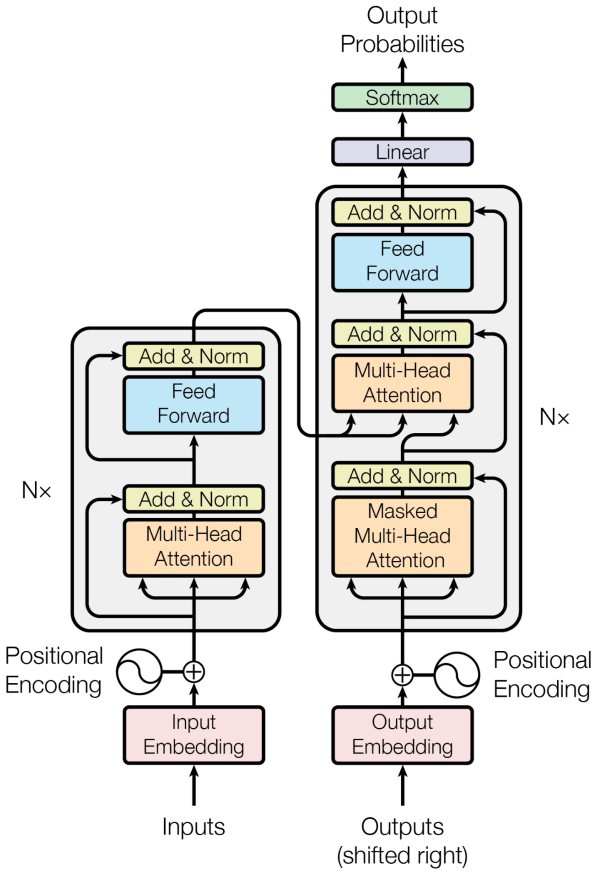
Transformers



source: Vaswani et al. "Attention is all you need", 2017

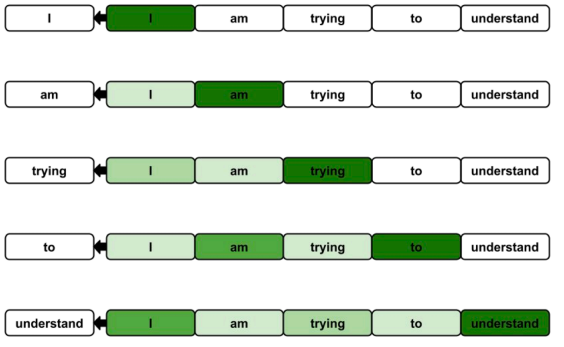
- Encoder (left branch): representation learning part
- Once trained, it can be used as a **pre-trained representation model** for many downstream tasks (classification, labeling, etc.)

Transformers



source: Vaswani et al. "Attention is all you need", 2017

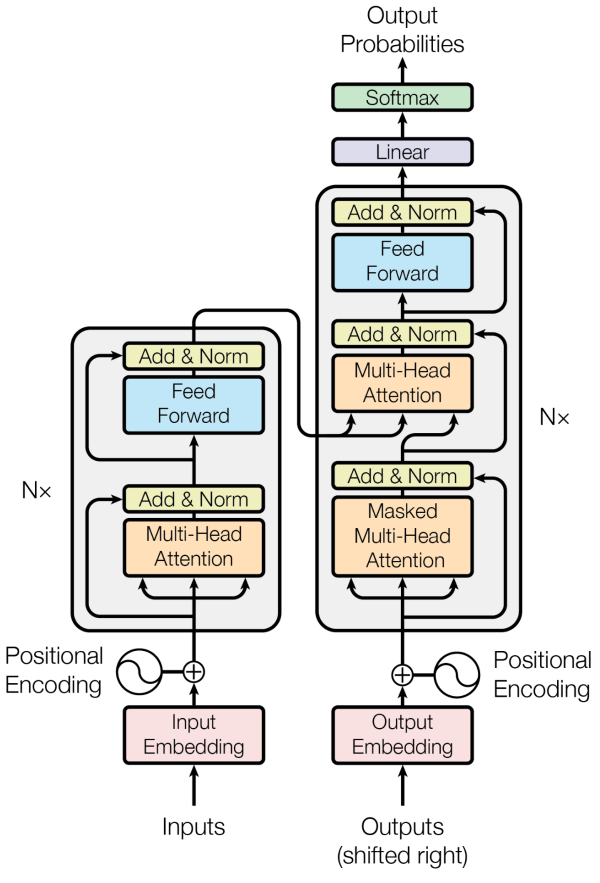
- Decoder (right branch): sequence generation part (thanks to unidirectional attention)



Unidirectional attention (source: fidle.cnrs.fr)

- It's the core part of **Large Language Models (LLM)** (e.g. the GPT model of ChatGPT)

Transformers



source: Vaswani et al. "Attention is all you need", 2017

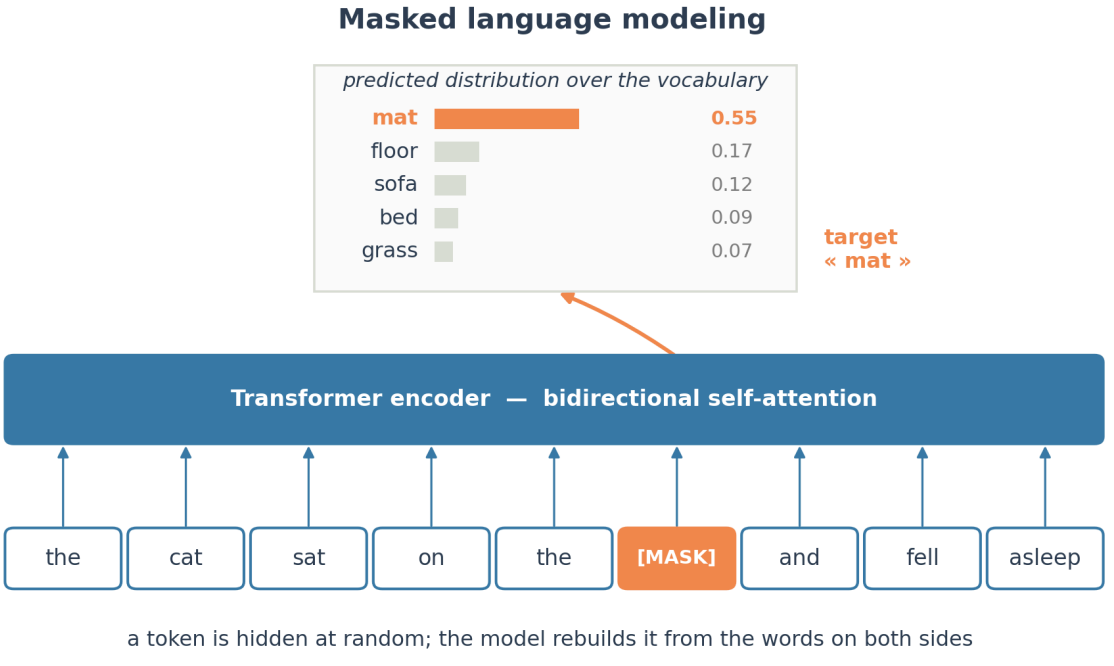
Few important tricks to make it work in practice:

- **Text embeddings**: map discrete symbols to continuous vectors
- **Positional encoding**: add information about the position of each element
- **Residual connections and layer normalization**: stabilize training
- **MLPs**: add non-linearity and increase model capacity

Self supervised training

- Now, the question is "how to train such a model to make it learn the semantics of language?"
- We have a lot of text available, but we don't have labels for it
- Solution: **self-supervised training**, i.e. create a supervised task from the text itself

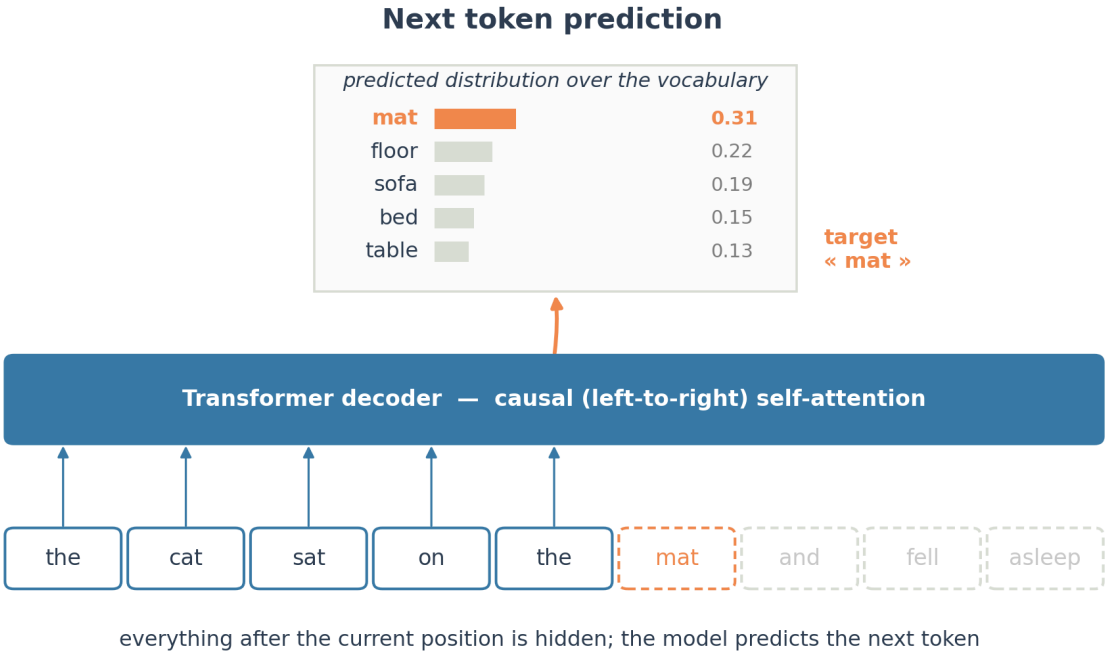
Example:



Self supervised training

- Now, the question is "how to train such a model to make it learn the semantics of language?"
- We have a lot of text available, but we don't have labels for it
- Solution: **self-supervised training**, i.e. create a supervised task from the text itself

Example:



Large Language Models

- Transformers have been to text processing what CNN have been to image processing
- **Large Language Models (LLM) are transformer-based foundations models**
- Pre-trained on HUGE corpus of text
- Today: in much more complex systems with enhanced capabilities (RAG, Multi-modal, conversational agents, etc.)



Takeaways

Sequences are variable-length data that can be discrete (text) or continuous (audio, signals).

Recurrent neural networks (RNN), designed to process sequences, have strong limitations in practice. LSTM and GRU cells are the best of the kind.

Attention mechanisms allow models to focus on relevant parts of the input sequence when generating outputs, enabling efficient sequence-to-sequence modeling.

Transformers, based on self-attention, have become the state-of-the-art architecture for sequence modeling, particularly in natural language processing. They are the foundation of Large Language Models (LLMs) like GPT.